

Android with Arduino

There must be a lot of people out there who love trying out things with Android, and who spend a lot of time experimenting with it. Here is something for those who love creating things with FOSS and electronic prototyping boards



YATHARTH A. KHATRI

Arduino is an Open Source, single-microcontroller electronics prototyping board with easy-to-use hardware and software. It was developed in 2005 by Massimo Banzi and David Cuartielles. Arduino is capable of interacting with the environment by receiving inputs from a broad range of sensors and responding by sending outputs to various actuators.

The Arduino board consists of 8-bit Atmel AVR microcontrollers. In addition, the board has a standard way of connecting the CPU with various other complementary components to increase its functionality through a number of add-ons called shields. You can either assemble an Arduino board yourself or purchase it (pre-assembled) from <http://arduino.cc/en/Main> depending on your needs.

Installing and working with Arduino

The microcontroller on the Arduino board is programmed via Arduino Programming Language (based on Wiring) and Arduino Development Environ-

ment (based on Processing). The Open Source Arduino environment can be downloaded for Windows, Linux or Mac OS X from <http://arduino.cc/en/Main/Software> and extracted. This makes writing the code and uploading it to the board very easy. As the environment is written in Java, make sure you have Java installed.

When you are finished with installation, start off with Arduino programming. When working with electronic

prototyping boards, all programmers must be familiar with 'Hello World' sample of physical computing—for microcontrollers that don't have a display device, an LED is added. So just start the Arduino software, select your board model and enter the following code:

```
int ledPin = 13;
// LED connected to digital pin 13

void setup() {
  pinMode(ledPin, OUTPUT);
// sets the digital pin as output
}

void loop() {
  digitalWrite(ledPin, HIGH);
// sets the LED on
  delay(1000);
// waits for a second
  digitalWrite(ledPin, LOW);
// sets the LED off
  delay(1000);
// waits for a second
}
```

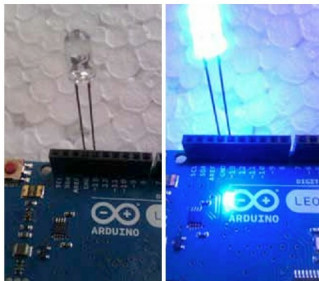


Fig. 1: The Arduino LED shown blinking

Once you've typed in the code, connect your board via USB, and upload the program to it. As the LED has polarity, you need to fix it onto the board carefully. The long leg, typically positive, should be connected to pin 13, and the short leg to GND (i.e., ground). The LED starts turning 'on' and 'off' at intervals of one second, as shown in Fig. 1.